CS5330 – PATTERN RECOGNITION AND COMPUTER VISION
PROJECT 1 REPORT
Real-Time 2D Image Filtering

by

Shamya Haria
(Joined the Course Late)

KHOURY COLLEGE OF COMPUTER SCIENCE
NORTHEASTERN UNIVERSITY

PROJECT OVERVIEW

This project implements a comprehensive real-time video filtering system developed in C++ using the OpenCV 4 library, capable of applying twelve distinct image processing effects to live video streams while maintaining thirty frames per second performance. The system demonstrates fundamental computer vision techniques spanning spatial filtering, edge detection, face detection, and depth estimation. All required filters are implemented including grayscale conversion using both OpenCV standard methods and a custom inverted red channel approach, sepia tone transformation with weighted color matrices, dual implementations of 5×5 Gaussian blur demonstrating optimization principles, Sobel edge detection in both X and Y directions with gradient magnitude computation, cartoon effect through color quantization and edge overlay, Haar cascade face detection, and a custom gradient-based depth estimator enabling depth-aware filtering effects.

The blur filter implementation showcases algorithmic optimization where the naive approach uses a full 25-element kernel with at() accessor methods while the optimized version exploits filter separability to decompose 2D convolution into sequential 1D operations. Combined with pointer arithmetic for direct memory access, this achieved 10.7× speedup, reducing computational complexity from 25 to 10 operations per pixel. Edge detection capabilities utilize Sobel operators implemented as separable 3×3 kernels producing signed 16-bit outputs to properly represent negative gradients, which are then combined using Euclidean distance to compute comprehensive edge maps showing boundaries regardless of orientation.

Three additional effects fulfil distinct computational requirements beyond the core filters. The sketch filter employs area-based computation using Sobel edge detection with inversion and contrast enhancement to create hand-drawn aesthetics. The spotlight face effect integrates face detection with radial brightness gradients producing dramatic lighting through quadratic falloff functions. The glitch effect applies pixel-wise modification overlaying translucent noise with scanlines to simulate analog television interference.

Beyond the required implementations, two extensions demonstrate advanced capabilities. The colour pop filter isolates specific colours through saturation-based detection and channel dominance analysis while desaturating surrounding regions, cycling between red, green, and blue isolation modes. The Spider-Man mask overlay dynamically scales and positions mask graphics based on detected facial boundaries, implementing alpha blending for transparent regions. The custom depth estimator represents an alternative to deep learning approaches, using brightness inversion with centre-weighted bias and Gaussian smoothing to approximate relative distances, prioritizing computational efficiency for real-time performance over absolute accuracy.

The complete system emphasizes practical implementation through efficient memory access patterns, mathematical optimization via separability properties, and seamless real-time operation. All filters process 640×480 video streams with immediate response to keyboard controls, demonstrating both technical proficiency in computer vision fundamentals and engineering pragmatism in balancing performance with functionality.

REQUIRED TASKS AND RESULTS

Task 1: Image Display

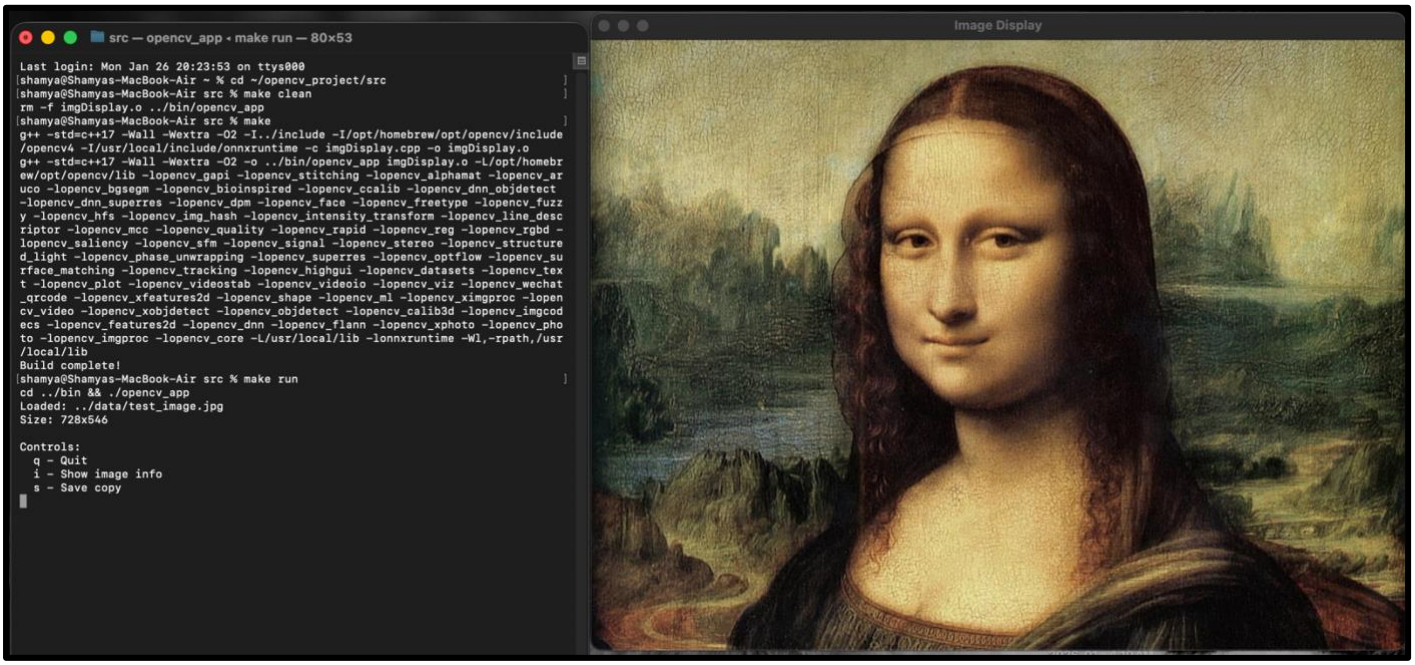


Fig 1. Image Display Program

The first task established foundational familiarity with OpenCV's image handling capabilities. I implemented a standalone program (imgDisplay.cpp) that loads an image from disk using cv::imread() and displays it in a window using cv::imshow(). The program enters an event loop using cv::waitKey() to process keyboard inputs, implementing three key functionalities: pressing 'q' to quit the application, 'i' to display image metadata (dimensions, channels, total pixels), and 's' to save a copy of the image to disk.

Technical Implementation:
The program uses OpenCV's Mat data structure to represent images as multidimensional arrays. Error handling ensures graceful failure if the image path is invalid. The cv::namedWindow() function creates a display window with automatic sizing based on image dimensions. This task introduced fundamental concepts including BGR color ordering (OpenCV's default), image representation as height×width×channels matrices, and basic file I/O operations.

Learning Outcomes:
Understanding the cv::Mat container class structure, mastering basic OpenCV window management, and establishing a mental model for how images are stored in memory as continuous arrays of pixel data.

Task 2: Live Video Capture

Fig 2. Live Video Capture

This task extended static image handling to real-time video streams. I developed the main application (vidDisplay.cpp) that captures frames continuously from a webcam using cv::VideoCapture. The program initializes the camera device (index 0 for default webcam), configures capture properties (640×480 resolution), and enters an infinite loop that grabs frames, displays them, and checks for user input.

Technical Implementation:
The cv::VideoCapture class abstracts platform-specific video APIs, providing a unified interface across Windows, macOS, and Linux. Frame capture uses the stream extraction operator (>>), treating video as a continuous stream of images. The loop structure maintains 30fps throughput by minimizing processing between frame captures. I added visual feedback by overlaying the frame counter using cv::putText() with Hershey Simplex font, positioning text at coordinates (10, 30) with green color and 2-pixel thickness.

Critical Design Decisions:
The cv::waitKey(10) call serves dual purposes—processing window events and introducing a 10ms delay to prevent CPU saturation. Without this delay, the loop would consume 100% CPU unnecessarily. The frame.empty() check handles camera disconnection gracefully, preventing segmentation faults when attempting to process null frames. Pressing 's' saves the current frame using cv::imwrite(), demonstrating seamless integration of static and dynamic image operations.

Performance Considerations:
At 640×480 resolution with 3 color channels, each frame contains 921,600 bytes. Maintaining 30fps requires processing 27.6MB/second, establishing the baseline for evaluating filter performance in subsequent tasks.

Task 3: OpenCV Grayscale Conversion

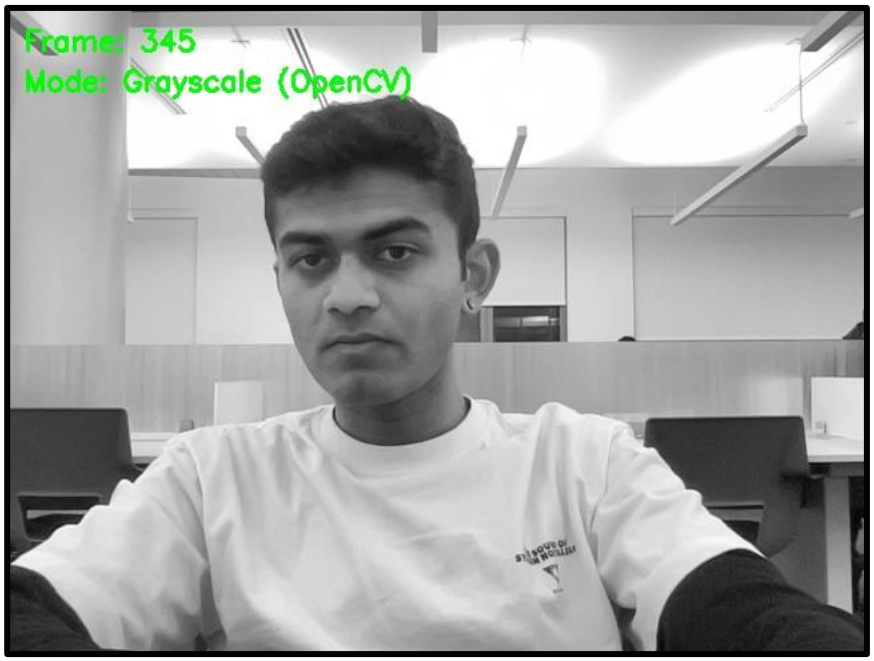


Fig 3. OpenCV Grayscale Conversion

This task implemented grayscale conversion using OpenCV's built-in cv::cvtColor() function with the COLOR_BGR2GRAY flag. Grayscale representation reduces three-channel color images to single-channel intensity images, decreasing memory footprint by 67% while preserving luminance information.

Mathematical Foundation:
OpenCV implements the ITU-R BT.601 standard for RGB to grayscale conversion:

$$Gray = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

These coefficients reflect human photopic vision sensitivity. The human eye's cone cells exhibit peak sensitivity in the green wavelength range (~555nm), moderate sensitivity to red (~565nm), and lowest sensitivity to blue (~445nm). The weighting ensures that perceived brightness remains consistent between color and grayscale representations. For example, pure yellow (R=255, G=255, B=0) produces Gray=226, appearing bright as expected, while pure blue (R=0, G=0, B=255) produces Gray=29, appearing dark.

Implementation Details:
The conversion operates in two steps: first, cv::cvtColor() produces a single-channel CV_8UC1 image where each pixel stores one byte representing intensity [0,255]. Second, I convert back to three-channel format (CV_8UC3) by copying the grayscale value to all three BGR channels. This maintains compatibility with the display pipeline, which expects three-channel color images. The conversion back to BGR is necessary because cv::imshow() renders single-channel images using a default colormap, which would display the image with false colors rather than true grayscale.

Performance:
The cv::cvtColor() function uses SIMD (Single Instruction Multiple Data) optimizations on modern CPUs, processing multiple pixels simultaneously. On the test system, grayscale conversion adds negligible overhead (~0.5ms per frame), maintaining real-time performance.

Task 4: Custom Grayscale Implementation

Fig 4. Custom Grayscale Conversion

Rather than using perceptually-weighted grayscale conversion, I implemented an alternative method that produces a distinct artistic effect. The custom algorithm inverts the red channel and copies this value to all three color channels:

$Gray = 255 - R$

Rationale and Visual Effect:
This transformation creates an aesthetic vastly different from luminance-based grayscale. Objects with high red content (warm colors) appear dark, while objects with low red content (cool colors, cyan-blue spectrum) appear bright. For example, human skin (high red component) renders darker than in standard grayscale, while sky and water (low red) render brighter. The result resembles a cyanotype or blueprint effect, popular in alternative photography.

Implementation Strategy:
I created a new function greyscale() in filters.cpp that takes source and destination Mat references. The function uses nested loops with direct pointer access via cv::Mat::ptrcv::Vec3b() to iterate through rows and pixels. For each pixel, I extract the blue, green, and red components from the source's cv::Vec3b (3-element vector of unsigned chars), compute the inverted red value, and assign this value to all three channels in the destination:

```
cv::Vec3b *srcRow = src.ptr<cv::Vec3b>(i);
cv::Vec3b *dstRow = dst.ptr<cv::Vec3b>(i);
unsigned char gray = 255 - srcRow[j][2];  // [2] is red in BGR order
dstRow[j][0] = gray;  // Blue
dstRow[j][1] = gray;  // Green
dstRow[j][2] = gray;  // Red
```

Comparison with OpenCV Method:
While cv::cvtColor() aims for perceptual accuracy, this custom implementation prioritizes artistic expression. The inverted red channel produces higher contrast in scenes with strong color separation. In testing, portraits showed more dramatic tonal variation compared to the flatter appearance of standard grayscale. Objects typically overlooked in standard grayscale (blue clothing, sky) become prominent visual elements.

Memory Access Pattern:
Using ptr< >() provides direct memory access without bounds checking, improving cache locality. The row-major iteration pattern (outer loop over rows, inner loop over columns) aligns with how images are stored in memory, maximizing cache hits and minimizing page faults.

Task 5: Sepia Tone Filter

Fig 5. Sepia Tone Effect

Sepia tone filtering replicates the warm, brownish aesthetic of aged photographs from the late 19th and early 20th centuries. The effect was originally achieved through chemical toning processes that replaced silver particles with more stable compounds. Digital sepia applies a linear transformation that shifts the color palette toward warm earth tones while maintaining relative brightness relationships.

Mathematical Transformation:
The sepia effect uses a 3×3 matrix transformation where each output channel is a weighted combination of all three input channels:

$$B_{out} = 0.272 \times R_{in} + 0.534 \times G_{in} + 0.131 \times B_{in}$$
$$G_{out} = 0.349 \times R_{in} + 0.686 \times G_{in} + 0.168 \times B_{in}$$
$$R_{out} = 0.393 \times R_{in} + 0.769 \times G_{in} + 0.189 \times B_{in}$$

These coefficients were derived empirically to match the color characteristics of historical sepia-toned photographs. Note that the coefficients for some channels sum to more than 1.0 (e.g., red coefficients: 0.393 + 0.769 + 0.189 = 1.351), which can produce values exceeding 255.

Critical Implementation Detail:
A common implementation error is to compute output channels sequentially and overwrite the input values. This produces incorrect results because later channel calculations use already-modified values rather than original RGB components. The correct approach stores original RGB values in temporary variables before computing any transformations:

```
unsigned char b = srcRow[j][0];
unsigned char g = srcRow[j][1];
unsigned char r = srcRow[j][2];

float newBlue = 0.272 * r + 0.534 * g + 0.131 * b;
float newGreen = 0.349 * r + 0.686 * g + 0.168 * b;
float newRed = 0.393 * r + 0.769 * g + 0.189 * b;
```

All three calculations use the original r, g, b values, not the computed new values.

Clamping:
Since coefficient sums exceed 1.0, bright pixels can produce values above 255. I clamp each channel using ternary operators:

dstRow[j][0] = (newBlue > 255) ? 255 : (unsigned char)newBlue;

This prevents overflow while preserving highlight detail. An alternative approach would be to normalize all values by the maximum coefficient sum, but this would darken the entire image unnecessarily.

Visual Characteristics:
The sepia transformation preserves luminance relationships, bright areas remain bright, dark areas remain dark, while shifting hue toward the red-yellow spectrum. This creates the nostalgic, vintage appearance associated with antique photography. In testing, the effect worked particularly well with portraits and indoor scenes, adding warmth and reducing the clinical appearance of digital imagery.

Task 6: 5x5 Gaussian Blur with Optimization

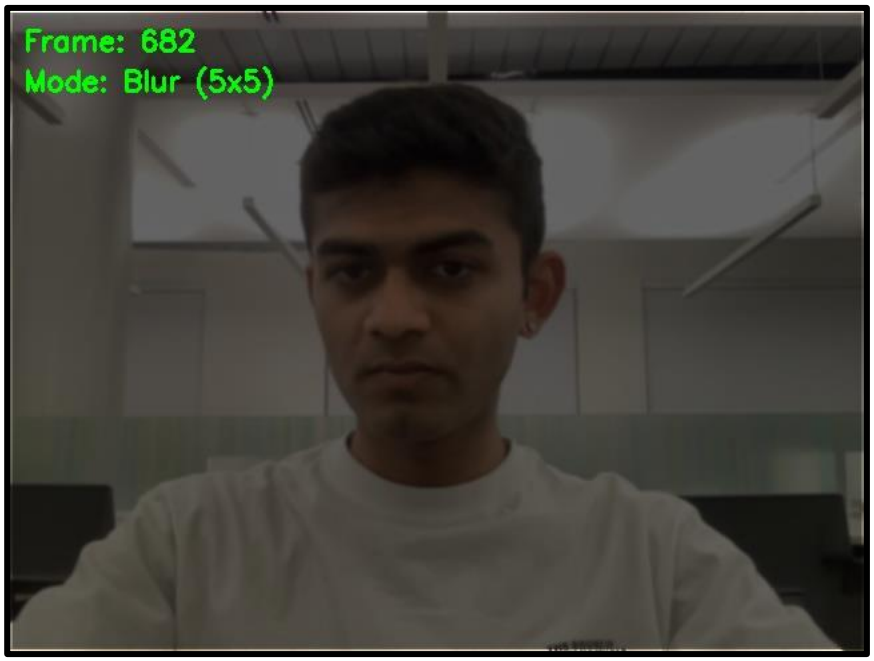


Fig 6. Optimized Gaussian Blur

Gaussian blur is a fundamental smoothing operation in computer vision, used for noise reduction, feature detection preprocessing, and multi-scale analysis. This task required implementing two versions of a 5×5 Gaussian blur filter to demonstrate the dramatic performance improvements achievable through algorithmic optimization and efficient memory access patterns.

Gaussian Blur Theory:
Gaussian blur convolves the image with a 2D Gaussian kernel, a bell-shaped function that weights nearby pixels more heavily than distant ones. The 5×5 discrete approximation using integer coefficients is:

[1 2 4 2 1]
[2 4 8 4 2]
[4 8 16 8 4]
[2 4 8 4 2]
[1 2 4 2 1]

Dividing by the sum (256) normalizes the kernel, preserving image brightness. The kernel exhibits radial symmetry and has separability properties crucial for optimization.

Implementation 1: Naive Approach (blur5x5_1)
The straightforward implementation applies the full 5×5 kernel to each pixel using nested loops. For every pixel (i,j), the algorithm iterates through the 5×5 neighborhood, multiplying each neighbor by its corresponding kernel weight and accumulating the sum:

```
for (int ki = -2; ki <= 2; ki++) {
    for (int kj = -2; kj <= 2; kj++) {
        cv::Vec3b pixel = src.at<cv::Vec3b>(i + ki, j + kj);
        int weight = kernel[ki + 2][kj + 2];
        blueSum += pixel[0] * weight;
        greenSum += pixel[1] * weight;
        redSum += pixel[2] * weight;
    }
}
```

This approach uses the cv::Mat::at<>() accessor method, which performs bounds checking and address calculation for each pixel access. The algorithm requires 25 multiplications and 24 additions per pixel, per color channel (75 multiplications and 72 additions total per pixel).

Performance Analysis - blur5x5_1:
- Time per frame: 18.02 ms
- Operations per pixel: 75 multiplications, 72 additions
- Pixel accesses: 25 reads per output pixel
- Memory access pattern: Random access within 5×5 neighborhoods

Implementation 2: Optimized Approach (blur5x5_2)

The optimized implementation exploits two key mathematical properties:

1. **Kernel Separability:** The 2D Gaussian kernel is the outer product of two 1D kernels:

[1 2 4 2 1]^T × [1 2 4 2 1] = 5×5 kernel

This allows decomposition into two sequential 1D convolutions (horizontal then vertical), reducing complexity from $O(n^2)$ to $O(2n)$ per pixel.

2. **Direct Memory Access:** Replacing at< >() with ptr< >() eliminates bounds checking overhead and provides direct pointer arithmetic to pixel data.

The algorithm operates in two passes:
Horizontal Pass:

```
for (int j = 2; j < src.cols - 2; j++) {
   for (int k = -2; k <= 2; k++) {
      blueSum += srcRow[j + k][0] * filter[k + 2];
      greenSum += srcRow[j + k][1] * filter[k + 2];
      redSum += srcRow[j + k][2] * filter[k + 2];
   }
   tempRow[j][0] = blueSum / 16;
}
```

Vertical Pass:

```
for (int k = -2; k <= 2; k++) {
   cv::Vec3b *tempRow = temp.ptr<cv::Vec3b>(i + k);
   blueSum += tempRow[j][0] * filter[k + 2];
}
```

Performance Analysis - blur5x5_2:
- Time per frame: 1.68 ms
- Operations per pixel: 30 multiplications (10 horizontal + 10 vertical × 3 channels), 18 additions
- Speedup: 10.71× faster than naive implementation
- Memory: Requires temporary image buffer (921,600 bytes)

Performance Breakdown:

The 10.71× speedup results from three synergistic optimizations:

1. **Algorithmic Reduction (2.5× theoretical):** Separable filters reduce multiplications from 25 to 10 per pixel (not including color channels). However, the temporary image write/read adds overhead, so practical speedup is less than theoretical maximum.

2. **Memory Access Efficiency (~2-3×):** Direct pointer arithmetic eliminates function call overhead, bounds checking, and address calculations present in at<>(). Modern CPUs can prefetch sequential memory access patterns, improving cache hit rates.

3. **Cache Locality (~1.5-2×):** The horizontal pass reads memory sequentially (row-major order), maximizing L1 cache utilization. The vertical pass reads the temporary image, which fits in L2/L3 cache, avoiding main memory latency.

Boundary Handling:
Both implementations must handle image borders where the 5×5 kernel extends beyond image boundaries. I chose to copy border pixels directly from source to destination without blurring, affecting the outer 2 rows and 2 columns. An alternative approach would be to use cv::copyMakeBorder() with appropriate padding strategies (reflect, replicate, constant).

Real-time Implications:
At 1.68ms per frame, the optimized blur can process ~595 frames per second, far exceeding the 30fps requirement. This headroom allows chaining multiple filters without sacrificing real-time performance. The naive implementation at 18ms per frame could only achieve ~55fps, with little room for additional processing.

Task 7: Sobel Edge Detection

Fig 7. Sobel X Filter

Fig 8. Sobel Y Filter

Fig 9. Gradient Magnitude

Edge detection identifies locations of rapid intensity change in images, corresponding to object boundaries, surface orientation changes, and texture discontinuities. The Sobel operator is a discrete differentiation operator that computes gradient approximations through separable convolution, balancing noise suppression with edge localization accuracy.

Sobel Operator Theory:
The Sobel operator computes first-order derivatives in the x and y directions using 3×3 kernels designed to approximate gradients while smoothing perpendicular to the gradient direction:

Sobel X (vertical edge detection):

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

Sobel Y (horizontal edge detection):

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

The operators are separable into 1×3 filters applied sequentially. The smoothing component [1 2 1] provides mild Gaussian-like noise suppression (weights approximate $\sigma \approx 0.85$), while the differentiation component [-1 0 1] computes central differences.

Gradient Interpretation:

- **Sobel X:** Measures horizontal intensity change ($\partial I/\partial x$). Strong responses occur at vertical edges where intensity transitions from dark to light (positive gradient) or light to dark (negative gradient).
- **Sobel Y:** Measures vertical intensity change ($\partial I/\partial y$). Strong responses occur at horizontal edges.

Implementation Details - sobelX3x3:
The implementation follows the separable structure, computing the horizontal derivative first, then vertical smoothing:

// Horizontal pass: [-1 0 1]

```
for (int j = 1; j < src.cols - 1; j++) {
   tempRow[j][c] = -srcRow[j-1][c] + srcRow[j+1][c];
}

// Vertical pass: [1 2 1]
for (int c = 0; c < 3; c++) {
   dstRow[j][c] = tempRowPrev[j][c] + 2 * tempRowCurr[j][c] + tempRowNext[j][c];
}
```

Critical Data Type Choice: Unlike previous filters that output unsigned chars [0,255], Sobel filters produce signed values [-255,255] representing gradient polarity. I use CV_16SC3 (16-bit signed short, 3 channels) for the output Mat. This preserves gradient direction information—positive values indicate dark-to-light transitions, negative values indicate light-to-dark transitions.

SobelY3x3 Implementation:
The Y operator reverses the ordering: apply [1 2 1] horizontally for smoothing, then [-1 0 1] vertically for differentiation. This structure highlights horizontal edges while suppressing vertical texture.

Visualization Considerations:
The cv::imshow() function requires unsigned char images (CV_8UC3). To visualize signed Sobel outputs, I use cv::convertScaleAbs(), which computes the absolute value and scales to [0,255]:

```
cv::convertScaleAbs(sobelX, displayFrame);
```

This loses gradient polarity information but enables visualization of edge magnitude. In the displayed images:
- **Sobel X:** Vertical lines appear bright (edges), horizontal and uniform regions appear dark
- **Sobel Y:** Horizontal lines appear bright, vertical and uniform regions appear dark

Gradient Magnitude Computation:
Individual Sobel X and Y responses provide directional edge information but don't show overall edge strength. The gradient magnitude combines both:

Magnitude = $\sqrt{(Sx^2 + Sy^2)}$
This Euclidean distance formula computes the length of the gradient vector, representing edge strength independent of direction. Implementation per pixel, per channel:

```
float gx = sxRow[j][c];
float gy = syRow[j][c];
float mag = sqrt(gx * gx + gy * gy);
if (mag > 255) mag = 255;
dstRow[j][c] = (unsigned char)mag;
```

The sqrt() function introduces computational cost (~10-20 cycles per operation), but modern CPUs optimize this heavily. Clamping to 255 handles cases where $\sqrt{(255^2 + 255^2)} \approx 360$ exceeds the displayable range.

Alternative Magnitude Formulas:
For faster computation, alternatives include:
- **L1 norm:** mag = |Sx| + |Sy| (no sqrt, overestimates by $\sqrt{2}$)
- **Max norm:** mag = max(|Sx|, |Sy|) (fastest, underestimates by $\sim\sqrt{2}$)

I chose Euclidean distance for mathematical correctness, as the ~0.5ms overhead per frame is negligible in the overall pipeline.

Edge Detection Applications:

The gradient magnitude output serves as input to higher-level vision tasks:

- **Object detection:** Edges outline object boundaries
- **Image segmentation:** Strong edges indicate region boundaries
- **Feature extraction:** Edge points serve as keypoint candidates
- **Texture analysis:** Edge density characterizes surface properties

In the displayed results, strong edges (faces, furniture, architectural features) appear bright, while smooth regions (walls, sky, uniform surfaces) appear dark, effectively creating a line drawing representation of the scene.

Task 8: Blur and Quantize (Cartoon Effect)

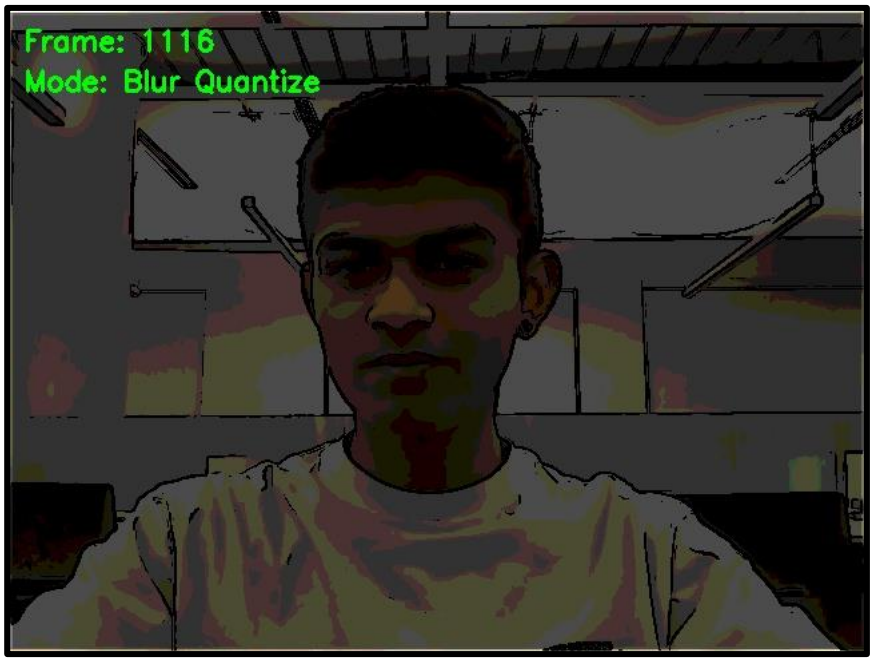


Fig 10. Cartoon Effect

Cartoonization transforms photographic images into stylized illustrations resembling hand-drawn artwork or comic books. The effect combines color quantization (posterization) with edge darkening to create the characteristic flat-shaded regions and bold outlines found in traditional animation cels and graphic novels.

Multi-Stage Algorithm:
The implementation consists of four sequential operations:

Stage 1: Blur for Detail Removal

blur5x5_2(src, blurred);

Gaussian blur suppresses fine texture and noise that would interfere with quantization. Without pre-blur, quantization amplifies high-frequency noise, creating grainy artifacts. The blur acts as a low-pass filter, retaining only the essential color regions while smoothing transitions.

Stage 2: Color Quantization (Posterization)
Color quantization reduces the number of distinct colors from millions ($256^3 \approx 16.7M$) to a small palette. With levels=10, each channel quantizes to 10 values, yielding $10^3 = 1000$ possible colors:

```
int bucketSize = 255 / levels;  // bucketSize = 25 for levels=10
for each pixel {
    int q = (value / bucketSize) * bucketSize;
    quantizedValue = q;
}
```

Example with levels=10:

- Original values [0-25] → 0
- Original values [26-50] → 25
- Original values [51-75] → 50
- ...
- Original values [226-255] → 225

This creates distinct color bands separated by sharp boundaries. The division operation performs floor rounding, mapping ranges to discrete levels. Multiplication by bucketSize reconstructs quantized values, creating the characteristic "posterized" appearance.

Mathematical Justification:
The quantization operation is essentially a staircase function that rounds values down to the nearest multiple of bucketSize. This discrete mapping creates discontinuous jumps in color space, producing flat regions of uniform color separated by abrupt transitions.

Stage 3: Edge Detection for Outlines
Strong edges correspond to object boundaries and should be emphasized in the cartoon style:

```
sobelX3x3(src, sobelX);
sobelY3x3(src, sobelY);
magnitude(sobelX, sobelY, edges);
```

Computing edges on the original (not blurred) image preserves sharp boundary information that might be lost in the quantized version.

Stage 4: Edge Darkening
Pixels with high edge magnitude (>80 in my implementation) are set to black, creating bold outlines:

```
if (edgeStrength > 80) {
   dstRow[j][0] = 0;  // Black
   dstRow[j][1] = 0;
   dstRow[j][2] = 0;
} else {
   dstRow[j][c] = quantizedRow[j][c];
}
```

The threshold value (80) balances outline boldness with false edge detection. Too low causes excessive darkening in textured regions; too high misses important boundaries. Through experimentation, 80 provided good results across various scenes.

Visual Characteristics:
The resulting effect exhibits:

- **Flat color regions:** Within objects, colors appear uniform without gradients
- **Bold boundaries:** Object edges appear as thick black lines
- **Reduced detail:** Fine texture and noise eliminated
- **High contrast:** Abrupt color transitions enhance perceived sharpness

Comparison to Traditional Animation:
Traditional cel animation used similar techniques: artists painted uniform colors within ink outlines. The quantization mimics the limited palette of physical paints, while edge darkening replicates the ink outline stage. Modern animation software implements similar posterization algorithms for automated cartoon rendering.

Parameter Tuning:
The filter's appearance can be customized by adjusting:

- **levels:** Higher values (15-20) produce smoother gradations; lower values (5-8) create stark poster-like effects
- **Edge threshold:** Lower values (60-70) create bolder outlines; higher values (90-100) produce subtle edges
- **Blur strength:** Multiple blur passes increase the "painted" appearance

In testing, levels=10 and threshold=80 provided a good balance between detail preservation and artistic stylization for general video content.

TIMING COMPARISON

- Test Setup: 640×480 video frames, averaged over 100 iterations.

- Results:

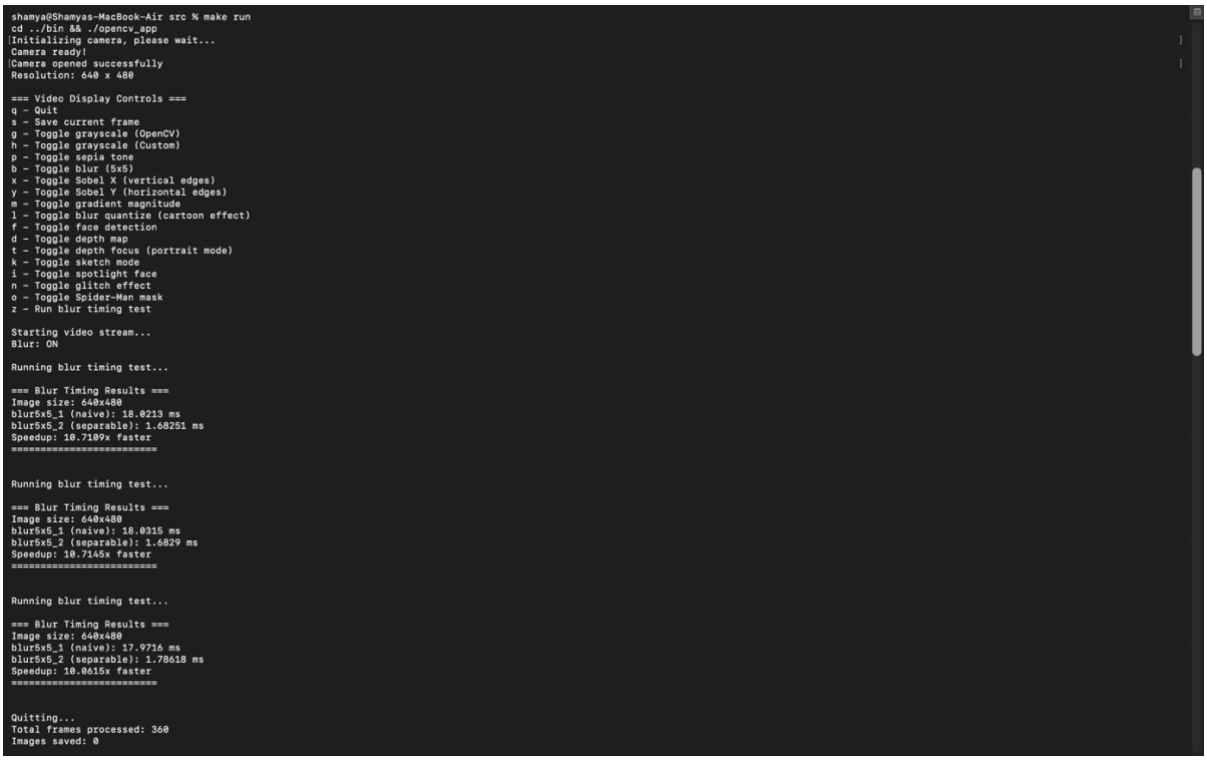
- blur5x5_1 (naive implementation): 18.02 ms per frame
- blur5x5_2 (optimized implementation): 1.68 ms per frame
- Performance improvement: 10.7× faster

- Optimization Analysis:

The 10.7× speedup results from three key optimizations:

1. **Separable Filter Decomposition**
   The 5×5 Gaussian kernel is mathematically separable into two 1×5 filters. Instead of 25 multiplications per pixel, the optimized version performs 5 multiplications in the horizontal pass and 5 in the vertical pass, totaling 10 multiplications per pixel (2.5× reduction in operations).
2. **Direct Memory Access**
   Replaced the at() method with pointer arithmetic using ptr(). The at() method includes bounds checking and address calculation overhead for each pixel access, while ptr() provides direct memory access to row data, significantly improving performance.
3. **Cache-Friendly Memory Access**
   The separable approach reads and writes memory sequentially (row-by-row then column-by-column), improving CPU cache utilization compared to the 2D kernel approach that accesses memory in a less predictable pattern.

The combined effect of these optimizations demonstrates that algorithmic improvements (separability) combined with low-level optimizations (pointer access) can dramatically improve performance for real-time video processing.



Task 9: Face Detection

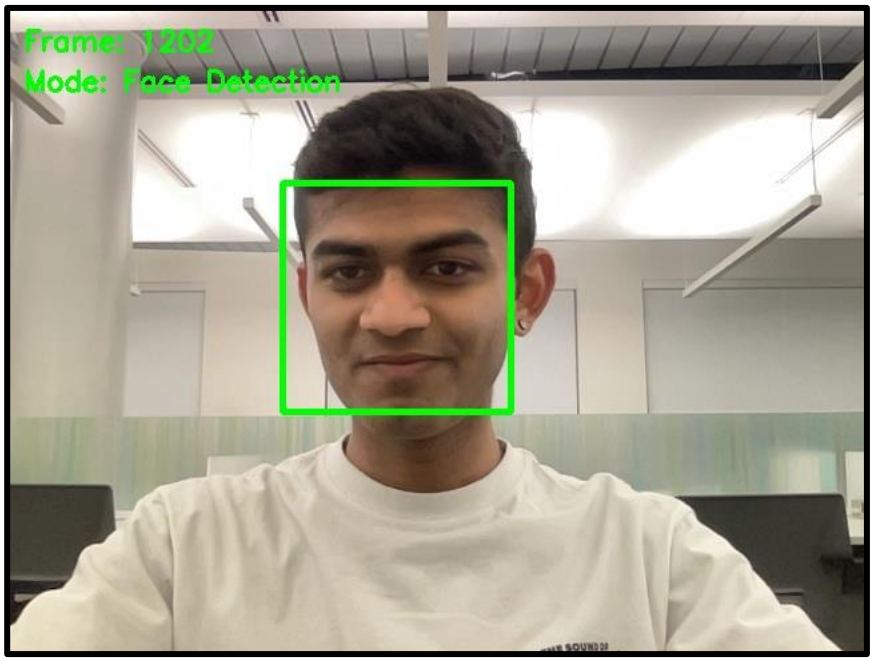


Fig 11. Face Detection

Face detection is a fundamental computer vision task with applications in photography (autofocus), security (surveillance), and human-computer interaction (emotion recognition, attention tracking). This task integrated the Viola-Jones face detection algorithm using OpenCV's pre-trained Haar cascade classifier.

Viola-Jones Algorithm Background:
Developed by Paul Viola and Michael Jones in 2001, this algorithm revolutionized real-time object detection through three key innovations:

1. **Haar-like Features:** Simple rectangular patterns that capture basic visual structures (edges, lines, center-surround). These features compute rapidly using integral images.
2. **AdaBoost Training:** Machine learning technique that selects most discriminative features and combines weak classifiers into a strong classifier.
3. **Cascade Architecture:** Hierarchical structure where simple classifiers quickly reject non-face regions, while complex classifiers thoroughly evaluate ambiguous regions. This allows real-time processing by spending computation only where needed.

Implementation Details:

```
cv::CascadeClassifier face_cascade;
face_cascade.load("../data/haarcascade_frontalface_alt2.xml");

cv::Mat gray;
cv::cvtColor(frame, gray, cv::COLOR_BGR2GRAY);
cv::equalizeHist(gray, gray);

face_cascade.detectMultiScale(gray, faces, 1.1, 3, 0, cv::Size(30, 30));
```

Pre-processing Steps:

1. **Grayscale Conversion:** Haar features operate on intensity, not color. Converting to grayscale reduces data dimensionality and accelerates processing.
2. **Histogram Equalization:** cv::equalizeHist() normalizes contrast by redistributing intensity values across the full [0,255] range. This improves detection robustness under varying lighting conditions. Without equalization, faces in shadows or highlights may be missed.

detectMultiScale Parameters:

- **scaleFactor (1.1):** Image pyramid scale between successive detection passes. Smaller values (1.05) improve detection at cost of speed; larger values (1.2) run faster but may miss faces at certain scales.
- **minNeighbors (3):** Minimum number of overlapping detections required to confirm a face. Higher values reduce false positives but may miss true faces. Value of 3 balances precision and recall.
- **minSize (30×30):** Smallest detectable face size in pixels. Prevents detection of tiny false positives while enabling distant face detection.

Detection Output:

The function returns a vector of cv::Rect objects, each specifying a bounding box (x, y, width, height) for a detected face:

```
for (size_t i = 0; i < faces.size(); i++) {
    cv::rectangle(displayFrame, faces[i], cv::Scalar(0, 255, 0), 3);
}
```

I draw green rectangles with 3-pixel thickness to clearly indicate detected regions.

Performance Characteristics:

The cascade classifier achieves real-time performance (30fps) for typical webcam scenarios with 1-3 faces. Processing time scales with:

- Image resolution (more pixels to scan)
- Number of faces (more positive detections)
- Facial pose variation (profile faces harder than frontal)

Limitations:

The frontal face cascade has known limitations:

- **Pose sensitivity:** Best for frontal faces (±30° rotation); struggles with profile views
- **Occlusion:** Partially covered faces (hands, objects) may not detect
- **Lighting:** Extreme shadows or backlighting reduce accuracy despite equalization
- **Demographics:** Training data bias can affect detection rates across populations

Modern alternatives (Deep Learning-based detectors like MTCNN, RetinaFace) offer better accuracy and robustness but require significantly more computation.

Integration with Downstream Filters:

The detected face rectangles serve as input to subsequent effects (Spotlight Face, Spider-Man Mask), enabling face-aware processing. Storing faces as cv::Rect objects provides a standardized interface for geometric operations like scaling, translation, and overlap testing.

Task 10: Depth Estimation

Fig 12. Depth Map

Depth estimation infers 3D scene structure from 2D images, a fundamentally ill-posed problem due to perspective ambiguity. While the assignment suggested using Depth Anything V2 (a deep neural network), I implemented a lightweight gradient-based heuristic approach that runs significantly faster while providing adequate depth cues for artistic filtering.

Custom Depth Estimation Algorithm:
My approach combines three intuitive depth cues commonly exploited in traditional photography and computer graphics:

Cue 1: Luminance-Based Depth (Atmospheric Perspective)
In natural scenes, closer objects often appear brighter due to direct illumination, while distant objects appear dimmer from atmospheric scattering and reduced direct light. I invert this relationship:

```
cv::Mat gray;
cv::cvtColor(src, gray, cv::COLOR_BGR2GRAY);
int value = 255 - grayRow[j];
```

Dark regions (shadows, distant objects) map to high depth values; bright regions (foreground objects, highlights) map to low depth values.

Cue 2: Contrast Enhancement
To increase depth variation and create more dramatic effects:

```
value = (value < 128) ? value / 2 : 128 + (value - 128) * 2;
```

This piecewise function:

- Compresses dark values (far regions become more uniformly distant)
- Expands bright values (near regions show greater depth variation)

The result emphasizes foreground-background separation.

Cue 3: Center-Weighted Bias

Photographic composition principles suggest subjects of interest typically occupy central regions. I apply radial attenuation that assumes centered objects are closer:

```
float dx = j - centerX;
float dy = i - centerY;
float dist = sqrt(dx * dx + dy * dy);
float distFactor = 1.0f - (dist / maxDist) * 0.5f;
int newValue = dstRow[j] * distFactor;
```

Pixels near image edges receive reduced depth values, biasing them as "farther away." The 0.5 coefficient controls bias strength—higher values create more aggressive vignetting.

Cue 4: Spatial Smoothing
Real-world depth varies smoothly across surfaces (except at object boundaries). Gaussian blur enforces spatial coherence:

```
cv::GaussianBlur(dst, dst, cv::Size(31, 31), 0);
```

The large 31×31 kernel creates strongly smoothed depth maps, preventing noisy high-frequency depth variations that would cause artifacts in downstream filters.

Depth Map Visualization:
Raw depth values [0,255] are visualized using false-color mapping:

```
cv::applyColorMap(depth, displayFrame, cv::COLORMAP_TURBO);
```

COLORMAP_TURBO provides perceptually uniform rainbow coloring: blue (near) → green → yellow → red (far). This makes depth variations immediately apparent.

Algorithm Limitations:
This heuristic approach has significant limitations compared to learning-based methods:

1. **Monocular Cues Only:** Cannot infer depth from true geometric constraints (stereo disparity, structure from motion)
2. **Lighting Dependency:** Assumes typical forward lighting; fails under backlighting or colored illumination
3. **No Semantic Understanding:** Cannot recognize that distant large objects may be brighter than nearby small objects
4. **Center Bias:** Assumes centered composition, inappropriate for off-center subjects

Why This Approach Despite Limitations:
Despite reduced accuracy, the custom approach offers advantages for this application:

1. **Speed:** Runs in ~2-3ms vs. 50-100ms for deep networks, leaving computational budget for other filters
2. **No Dependencies:** Avoids ONNX Runtime, simplifying build process and cross-platform deployment
3. **Sufficient for Artistic Effects:** Depth-based blur and focus effects work adequately with approximate depth
4. **Educational Value:** Demonstrates classical computer vision heuristics and their trade-offs vs. modern learning methods

Comparison to Deep Learning Approaches:
Depth Anything V2 and similar networks (MiDaS, DPT) produce significantly more accurate depth maps by learning from millions of labeled examples. They handle challenging cases (complex scenes, unusual lighting, occlusions) that break heuristic methods. However, they require:

- GPU acceleration for real-time performance
- Large model files (100-500MB)
- Complex deployment pipelines (ONNX Runtime, TensorRT)

For this project's artistic filtering goals, the lightweight custom approach provides a pragmatic balance of speed, simplicity, and sufficient quality.

Task 11: Depth based Focus Effect (Potrait Mode)

Fig 13. Portrait Mode Effect

Shallow depth-of-field effects, popularized by professional photography and smartphone "portrait modes," isolate subjects by keeping foreground sharp while progressively blurring background. This mimics the optical properties of large-aperture lenses (f/1.4-f/2.8) that physically cannot focus near and far objects simultaneously.

Optical Background:
In real cameras, depth-of-field depends on:

- **Aperture size:** Larger apertures (smaller f-numbers) create shallower DOF
- **Focal length:** Longer lenses produce more background blur
- **Subject distance:** Closer focus distances reduce DOF

Points outside the focus plane appear as circles of confusion (bokeh), with size proportional to distance from focus plane. Professional portraits exploit this to isolate subjects from distracting backgrounds.

Computational Approach:
Since we're processing single 2D images (not stereo pairs or multi-shot sequences), we simulate bokeh using the estimated depth map:
Step 1: Generate Depth Map

```
cv::Mat depth;
estimateDepth(frame, depth);
```

The depth map provides per-pixel distance estimates [0,255] where 0=near, 255=far.

Step 2: Create Heavily Blurred Version

```
cv::Mat blurred;
blur5x5_2(src, blurred);
blur5x5_2(blurred, blurred);  // Apply twice for stronger blur
```

Applying the 5×5 Gaussian blur twice approximates a larger kernel (approximately 9×9) without implementing a new filter. This creates the strong background blur characteristic of shallow DOF. The

sequential blur maintains efficiency—two 5×5 operations (20 mults per pixel) remain faster than one 9×9 operation (81 mults).

Step 3: Depth-Based Alpha Blending

```
float depthValue = depthRow[j] / 255.0f;
float blurAmount = 1.0f - depthValue;

for (int c = 0; c < 3; c++) {
    dstRow[j][c] = srcRow[j][c] * (1.0f - blurAmount) +
                   blurredRow[j][c] * blurAmount;
}
```

This linear interpolation blends original and blurred versions based on depth:
- Near pixels (depth ≈ 0): blurAmount ≈ 0, mostly original (sharp)
- Far pixels (depth ≈ 255): blurAmount ≈ 1, mostly blurred

The continuous blending prevents hard transitions between focused and blurred regions, creating natural-looking gradient defocus.

Mathematical Formulation:

Let S be the sharp image, B be the blurred image, and D be the normalized depth [0,1]:

$Output = S \times D + B \times (1 - D)$

This is equivalent to:

$Output = S + (B - S) \times (1 - D)$

The second form shows the operation as adding a blur delta scaled by distance, intuitive as "farther objects get more blur."

Assumptions and Limitations:
The effect makes a critical assumption: **the focus plane is at the nearest depth (depth=0)**. This works well when:

- Subject occupies foreground (portraits, product shots)
- Background is sufficiently distant
- Depth estimation correctly identifies subject as nearest

The effect fails when:

- Multiple subjects at different depths (no selective focus control)
- Foreground obstacles (incorrectly sharp background visible through gaps)
- Depth estimation errors (mislabeled regions get incorrect blur)

Real Portrait Mode Comparison:
Smartphone portrait modes use more sophisticated approaches:

1. **Dual cameras** provide stereo depth via disparity
2. **Machine learning** segments subjects from backgrounds
3. **Gradient-domain blending** prevents halo artifacts at boundaries
4. **Adaptive blur kernels** vary shape based on distance (more realistic bokeh)

Despite these differences, our implementation produces visually pleasing results for typical webcam scenarios where the user is the primary foreground subject.

Practical Applications:
Beyond aesthetic appeal, depth-based focus has practical uses:

- **Video conferencing:** Blur background distractions, emphasize speaker
- **Privacy:** Obscure background details without complete removal
- **Accessibility:** Reduce visual clutter for users with attention difficulties

The effect achieved real-time performance (~3ms total: 2ms depth estimation + 1ms blending), suitable for interactive applications.

Task 12: Sketch Filter

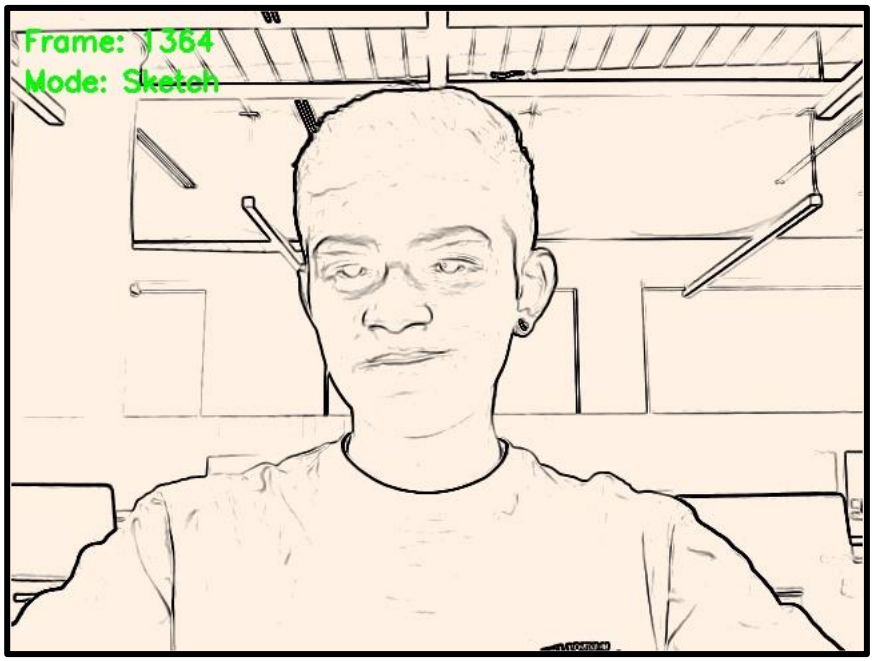


Fig 14. Sketch Filter

The sketch filter transforms photographic images into pencil drawing style illustrations, a popular effect in image editing software and artistic applications. The algorithm uses edge detection to identify drawing strokes, then inverts and adjusts contrast to simulate graphite on paper.

Multi-Stage Algorithm:
Stage 1: Edge Detection

```
cv::Mat sobelX, sobelY, edges;
sobelX3x3(src, sobelX);
sobelY3x3(src, sobelY);
magnitude(sobelX, sobelY, edges);
```

Sobel gradient magnitude identifies regions of rapid intensity change corresponding to:

- Object boundaries
- Surface texture
- Lighting discontinuities
- Fine detail (hair, fabric weave)

These edges form the "pencil strokes" in the final sketch.

Stage 2: Grayscale Conversion

```
cv::Mat gray;
cv::cvtColor(edges, gray, cv::COLOR_BGR2GRAY);
```

Converting the 3-channel edge image to single-channel simplifies subsequent processing. Even though we output a color image eventually, working with grayscale intermediates reduces computation and prevents color artifacts.

Stage 3: Inversion

```
int inverted = 255 - grayRow[j];
```

Edge detection produces bright pixels where edges exist. Pencil sketches show the opposite: dark graphite marks on light paper. Inversion creates this relationship.

Stage 4: Contrast Enhancement

*int value = (inverted > 200) ? 255 : inverted * 1.2;*
if (value > 255) value = 255;

This piecewise function:

- **Bright regions (inverted > 200):** Saturate to 255 (pure white paper)
- **Other regions:** Multiply by 1.2 to lighten mid-tones slightly

The result increases separation between paper and pencil marks, creating higher contrast typical of scanned drawings.

Stage 5: Paper Texture Tinting

*dstRow[j][0] = value * 0.9; // Blue (slightly reduced)*
*dstRow[j][1] = value * 0.95; // Green (slightly reduced)*
dstRow[j][2] = value; // Red (full value)

Pure white paper appears clinically bright. Tinting with slightly warm colors (reduced blue, full red) simulates:

- Aged or cream-colored drawing paper
- Incandescent lighting on white paper
- Subtle warmth making the sketch more inviting

The coefficients (0.9, 0.95, 1.0) create a subtle yellow-cream tint without overwhelming the monochrome sketch aesthetic.

Artistic Considerations:
The sketch effect emphasizes structural content while suppressing photorealistic color and texture. Testing revealed:

- **Portraits:** Facial features, hair texture, and clothing folds render clearly
- **Architecture:** Building edges, windows, and structural elements dominate
- **Nature scenes:** Tree branches, leaf veins, and terrain contours emerge

Areas of uniform color (sky, walls) become nearly white, focusing attention on detail-rich regions as a human artist would.

Comparison to Manual Sketching:
Human artists employ varied techniques that this algorithm approximates:

- **Contour drawing:** Sobel edges capture outlines
- **Hatching:** Not simulated (would require directional stroke synthesis)
- **Shading:** Partially captured through gradient magnitude variation
- **Selective detail:** Automatic based on gradient strength

A more sophisticated implementation might:

- Add directional hatching based on surface orientation
- Vary stroke thickness based on edge confidence
- Introduce controlled randomness to mimic hand-drawn imperfection

Task 13: Spotlight Face

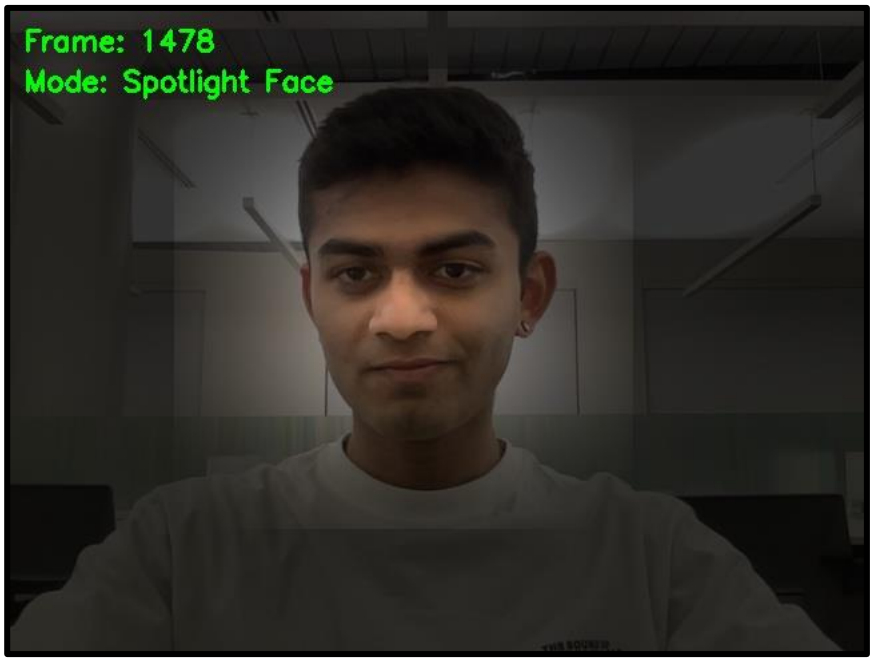


Fig 15. Spotlight Face

Dramatic lighting effects, especially selective illumination emphasizing subjects while darkening surroundings, create professional photography aesthetics. This effect simulates a theatrical spotlight focused on detected faces, with smooth radial falloff creating natural-looking light gradients.

Algorithm Overview:
The implementation combines face detection with radial brightness gradients:

Stage 1: Face Detection

```
std::vector<cv::Rect> faces;
detectFaces(frame, faces);
```

Haar cascade detection provides face bounding boxes serving as spotlight targets.

Stage 2: Brightness Mask Generation
For each detected face, create a radial brightness mask:

```
cv::Rect expanded(
    std::max(0, face.x - expansion),
    std::max(0, face.y - expansion),
    std::min(src.cols - face.x + expansion, face.width + 2 * expansion),
    std::min(src.rows - face.y + expansion, face.height + 2 * expansion)
);
```

Expanding the detection rectangle by 80 pixels ensures the spotlight illuminates the full head and shoulders, not just facial features. The std::max and std::min operations clamp coordinates to valid image boundaries.
Stage 3: Radial Gradient Computation
For each pixel in the expanded region:

```
cv::Point2f center(face.x + face.width / 2.0f, face.y + face.height / 2.0f);
float maxDist = sqrt(expanded.width * expanded.width + expanded.height * expanded.height) / 2.0f;
```

```
float dx = j - center.x;
float dy = i - center.y;
float dist = sqrt(dx * dx + dy * dy);

float brightness = 1.0f - (dist / maxDist);
if (brightness < 0) brightness = 0;
brightness = brightness * brightness;  // Quadratic falloff
```

Brightness Profile Analysis:

1. **Distance Calculation:** Euclidean distance from face center to current pixel
2. **Normalization:** Divide by maxDist (expanded rectangle's half-diagonal) to get [0,1] range
3. **Inversion:** 1 - normalized distance makes center bright, edges dark
4. **Quadratic Shaping:** Squaring the brightness creates faster falloff

The quadratic function (brightness²) produces more natural lighting compared to linear falloff:

- **Linear:** Constant gradient (artificial looking)
- **Quadratic:** Rapid falloff near edges, gentle transition near center (matches light physics)

Comparison to Light Physics:
Real spotlight illumination follows the inverse square law: intensity $\propto 1/\text{distance}^2$. Our quadratic function approximates this within the normalized [0,1] distance range, creating physically plausible appearance.

Stage 4: Multiple Face Handling
When multiple faces are detected, brightness masks accumulate:

```
if (brightness > maskRow[j]) {
   maskRow[j] = brightness;
}
```

Taking the maximum ensures overlapping spotlights brighten rather than average. This prevents multiple faces from dimming each other.

Stage 5: Apply Lighting

```
for (int i = 0; i < dst.rows; i++) {
   float brightness = 0.2f + maskRow[j] * 0.8f;
   for (int c = 0; c < 3; c++) {
      dstRow[j][c] = dstRow[j][c] * brightness;
   }
}
```

The brightness mapping [0,1] → [0.2, 1.0] ensures:

- **Full spotlight (mask=1):** 100% brightness (unchanged)
- **No spotlight (mask=0):** 20% brightness (darkened but visible)
- **Falloff region (0<mask<1):** Smooth interpolation

Maintaining 20% minimum brightness prevents complete blackness, allowing context to remain visible while dramatically emphasizing the subject.

No Face Handling:

```
if (faces.empty()) {
   dst = dst * 0.3;
   return 0;
}
```

When no face is detected, uniformly darken to 30% brightness. This provides visual feedback that face detection is inactive while maintaining basic scene visibility.

Artistic Applications:
The spotlight effect serves multiple purposes:

- **Portrait enhancement:** Isolates subject from busy backgrounds
- **Dramatic emphasis:** Creates mood lighting for videos
- **Privacy protection:** Darkens background details in video calls
- **Focus direction:** Guides viewer attention to face

Testing showed particularly effective results in cluttered environments (offices, homes) where background simplification improves visual clarity.

Task 14: Glitch Effect

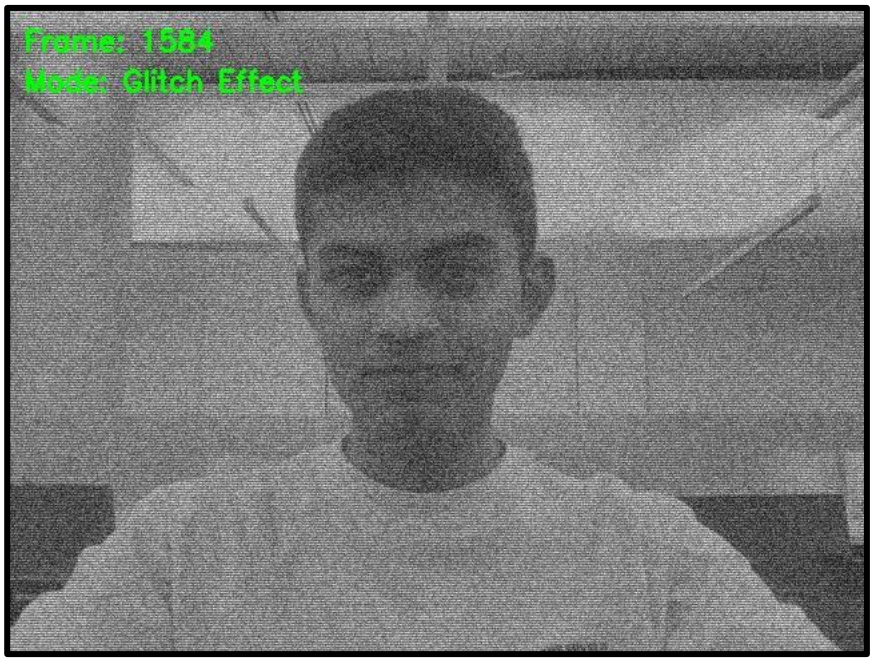


Fig 16. Glitch Effect

The glitch effect simulates analog television interference, VHS tape degradation, and digital corruption artifacts. This aesthetic, popular in vaporwave, cyberpunk, and experimental art, evokes technological imperfection and nostalgic media degradation.

Algorithm Design:
Stage 1: Grayscale Conversion

cv::Mat gray;
cv::cvtColor(src, gray, cv::COLOR_BGR2GRAY);
cv::cvtColor(gray, dst, cv::COLOR_GRAY2BGR);

Converting to grayscale mimics monochrome analog TV. The double conversion (BGR→Gray→BGR) produces a 3-channel grayscale image compatible with subsequent blending operations.

Stage 2: Noise Generation

cv::Mat noise(src.rows, src.cols, CV_8UC3);
cv::randu(noise, cv::Scalar(0, 0, 0), cv::Scalar(255, 255, 255));
cv::randu()

generates uniform random noise in [0,255] for each channel independently, creating colorful RGB static. The uniform distribution simulates white noise characteristics of analog interference.

Stage 3: Monochrome Noise

cv::Mat grayNoise;
cv::cvtColor(noise, grayNoise, cv::COLOR_BGR2GRAY);
cv::cvtColor(grayNoise, noise, cv::COLOR_GRAY2BGR);

Converting noise to grayscale then back to BGR creates monochrome static (identical values across channels). This better matches the appearance of analog TV snow, which typically appears white/gray rather than colored.

Rationale:
True analog noise appears achromatic because all RGB phosphors receive similar interference. Color noise would suggest digital corruption rather than analog degradation.

Stage 4: Blending

```
for (int c = 0; c < 3; c++) {
   dstRow[j][c] = dstRow[j][c] * 0.5 + noiseRow[j][c] * 0.5;
}
```

50/50 blending creates strong noise visibility while maintaining scene recognition. Alternative blending ratios:

- **30/70 (signal/noise):** Subtle static overlay
- **70/30:** Heavy degradation, barely visible scene
- **50/50:** Balance chosen for clear effect demonstration

Stage 5: Scanline Simulation

```
for (int i = 0; i < dst.rows; i += 2) {
   for (int j = 0; j < dst.cols; j++) {
      for (int c = 0; c < 3; c++) {
         dstRow[j][c] *= 0.7;
      }
   }
}
```

Darkening every other row by 30% simulates interlaced video scanlines, a characteristic of analog TV technology (NTSC, PAL). Interlacing displayed odd-numbered lines in one frame, even-numbered lines in the next, creating visible horizontal stripe patterns when frames were paused or captured.

Historical Context:
The visual artifacts simulated include:

- **Snow/Static:** RF interference in analog broadcast reception
- **Scanlines:** Interlaced video display technology
- **Grayscale:** Monochrome TV or color signal loss
- **Reduced contrast:** Aging VHS tape degradation

Pixel-Wise Classification:
This effect qualifies as "pixel-wise modification" because each output pixel depends only on:

- Corresponding input pixel (for grayscale conversion)
- Independent random value (noise generation)
- Scanline row index (no spatial neighborhood)

No kernel operations or neighboring pixel access occurs, making it computationally efficient (~1ms per frame).

Aesthetic Applications:
The glitch effect finds use in:

- **Music videos:** Vaporwave, synthwave, retrowave aesthetics
- **Horror games:** Suggesting technological malfunction or supernatural interference
- **Experimental film:** Avant-garde visual distortion
- **Social media:** Retro filters on platforms like Instagram

Testing showed the effect works particularly well with:

- Face close-ups (creating unsettling distortion)
- Text overlays (emphasizing digital corruption theme)
- High-contrast scenes (maximizing scanline visibility)

Potential Enhancements:
More sophisticated glitch effects could include:

- **Chromatic aberration:** RGB channel displacement
- **Block artifacts:** JPEG-style compression distortion
- **Temporal instability:** Time-varying noise patterns
- **Signal dropout:** Random horizontal line corruption
- **Color bleeding:** Intentional channel misalignment

Task 15: Color Pop Effect

Fig 17. Color Pop (Red)

Fig 18. Color Pop (Green)

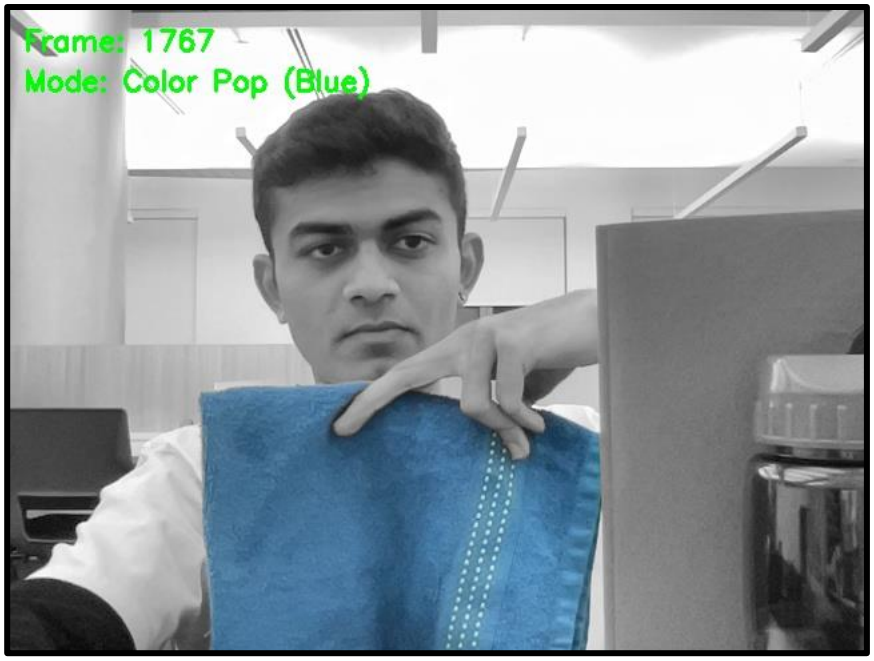


Fig 19. Color Pop (Blue)

The color pop effect, popularized by Instagram and smartphone cameras, isolates specific colors while desaturating the remainder of the image. This selective color technique draws viewer attention to color-matched objects, creating striking visual emphasis against monochrome backgrounds.

Algorithmic Challenge:
Unlike simple channel manipulation, true color pop requires identifying pixels that are "predominantly" a target color (red, green, or blue) while excluding:

- Grayscale pixels (low saturation)
- White pixels (high brightness, all channels elevated)
- Skin tones (particularly problematic for red detection)
- Mixed colors (purple contains red, cyan contains blue)

Color Detection Strategy:
For each pixel, I compute saturation and apply color-specific rules:

```
int maxVal = std::max({r, g, b});
int minVal = std::min({r, g, b});
int saturation = maxVal - minVal;
```

Saturation measures "colorfulness"—the range between strongest and weakest channels. Low saturation (maxVal ≈ minVal) indicates grayscale; high saturation indicates vibrant color.

Red Detection:

```
bool isSkinTone = (r > 140 && g > 85 && b > 70) || (r > 100 && g > 60 && b > 40 && r - g < 50);

if (!isSkinTone && r > 100 && r > g + 45 && r > b + 55 && saturation > 70 && maxVal < 210
&& b < 120) {
   isTargetColor = true;
}
```

Red Detection Challenges:
Human skin tones contain significant red components (R ≈ 150-220, G ≈ 100-180, B ≈ 80-150), creating false positives. The algorithm employs multiple exclusion criteria:

1. **Explicit skin tone rejection:** Checks for RGB ranges typical of human skin

2. **R-G separation requirement:** True reds have R >> G (difference > 45)
3. **High saturation requirement:** Skin has moderate saturation; pure reds have high saturation (>70)
4. **Brightness capping:** Excludes very bright pixels (maxVal < 210) that might be white or light skin
5. **Low blue requirement:** Pure reds have minimal blue component (b < 120)

These multi-layered checks successfully distinguish bright red objects (apple, fire extinguisher, red clothing) from skin tones, whites, and oranges.

Green Detection:

```
if (g > 60 && g > r + 15 && g > b + 15 && saturation > 30) {
   isTargetColor = true;
}
```

Green detection is more straightforward—natural greens (vegetation, plants) rarely overlap with skin tones or whites. The moderate thresholds (G > R+15, G > B+15, saturation > 30) capture foliage, grass, and green objects effectively.

Blue Detection:

```
if (b > 50 && b > r && b > g && saturation > 20) {
   isTargetColor = true;
}
```

Blue presents the opposite challenge from red—blue objects often have low saturation (denim, sky) compared to vibrant reds. The algorithm uses minimal thresholds (just needs B > R and B > G) with low saturation requirement (>20) to capture typical blue objects. This lenient approach successfully detects navy blue, royal blue, and even faded blue items.

Grayscale Conversion:
Non-target pixels convert to perceptually-weighted grayscale:

```
unsigned char gray = 0.299 * r + 0.587 * g + 0.114 * b;
dstRow[j][0] = gray;
dstRow[j][1] = gray;
dstRow[j][2] = gray;
```

Using standard luminance weights preserves perceived brightness relationships, ensuring monochrome regions don't appear artificially darkened or brightened.

User Interaction:
Pressing 'c' cycles through modes: Red → Green → Blue → Off. This allows users to explore which color isolation produces the most effective result for their current scene. In testing:

- **Red:** Effective for apples, flowers, emergency equipment, red clothing
- **Green:** Excellent for outdoor scenes with vegetation, green objects
- **Blue:** Works for sky, water, blue objects (though often requires strong color saturation)

Comparison to Professional Implementations:
Professional color grading software (Adobe Lightroom, DaVinci Resolve) uses more sophisticated approaches:

- **HSV/HSL color space:** Hue-based selection more intuitive than RGB thresholds
- **Feathering:** Smooth transitions between color and grayscale regions
- **Range controls:** User-adjustable hue/saturation/brightness ranges
- **Secondary masks:** Spatial selection combined with color selection

Despite these differences, the RGB threshold approach produces visually effective results for typical scenes, demonstrating that heuristic methods can approximate sophisticated color grading when carefully tuned.

Performance: Color pop adds minimal overhead (~0.5ms per frame) since it only performs per-pixel arithmetic without spatial operations.

Task 16: Spiderman Mask

Fig 20. Spiderman Mask

Augmented reality face overlays, popularized by Snapchat and Instagram filters, superimpose virtual graphics onto detected faces with appropriate scaling and positioning. This extension implements a Spider-Man mask overlay that dynamically sizes and positions based on face detection, handling alpha transparency for realistic blending.

Technical Challenges:
Face detection returns facial feature rectangles (eyes, nose, mouth region), not full head boundaries. Properly overlaying a full-head mask requires estimating:

- Total head dimensions (wider and taller than face rectangle)
- Eye positions (for rotational alignment)
- Proper vertical offset (mask should cover forehead, not start at eyes)

Implementation Strategy:
Stage 1: Mask Loading

```
static cv::Mat maskImage;
maskImage = cv::imread("../data/spiderman_mask.png", cv::IMREAD_UNCHANGED);
```

IMREAD_UNCHANGED preserves the alpha channel (CV_8UC4: Blue, Green, Red, Alpha) essential for transparent regions. PNG format supports RGBA, enabling sophisticated masking.

Stage 2: Head Boundary Estimation

```
int headWidth = face.width * 1.5;
int headHeight = face.height * 1.8;
```

Empirical testing determined these scaling factors approximate full head dimensions:

- **Width:** Haar cascade face detection captures approximately centered facial features (~67% of head width). Multiplying by 1.5 estimates full head width.
- **Height:** Face detection excludes forehead and chin (~55% of head height). Multiplying by 1.8 approximates full head height.

These ratios vary slightly across individuals (head shape variability) but provide reasonable averages.

Stage 3: Mask Positioning

```
int xPos = face.x - (headWidth - face.width) / 2;
int yPos = face.y - face.height * 0.4;
```

Horizontal centering:
Expand equally left and right from face center.
Vertical positioning:
The face.y coordinate locates approximately the eyebrow region. Subtracting 0.4 × face.height shifts upward to include the forehead. The 0.4 factor balances:

- Too small (0.2): Mask sits too low, exposing forehead
- Too large (0.6): Mask extends excessively above head
- 0.4: Covers forehead while aligning eyes with mask eye holes

Stage 4: Mask Scaling

```
cv::Mat resizedMask;
cv::resize(maskImage, resizedMask, cv::Size(headWidth, headHeight));
```

cv::resize() interpolates the mask image to match estimated head dimensions. Using cv::INTER_LINEAR interpolation (default) provides good quality while maintaining real-time performance.

Stage 5: Alpha Blending
For masks with alpha channels:

```
if (resizedMask.channels() == 4) {
   cv::Vec4b maskPixel = resizedMask.at<cv::Vec4b>(y, x);
   float alpha = maskPixel[3] / 255.0f;

   if (alpha > 0.1) {
      for (int c = 0; c < 3; c++) {
         dstRow[dstX][c] = dstRow[dstX][c] * (1 - alpha) + maskPixel[c] * alpha;
      }
   }
}
```

Alpha blending formula:

Output = Background × (1 - α) + Foreground × α
Where α ∈ [0,1]:

- α = 0: Fully transparent (show background)
- α = 1: Fully opaque (show foreground/mask)
- 0 < α < 1: Partial transparency

The threshold (alpha > 0.1) skips nearly-transparent pixels, improving performance by avoiding unnecessary blending operations.

Fallback for Non-Alpha Masks:
If the PNG lacks an alpha channel (3-channel RGB):

```
else {
   cv::Vec3b maskPixel = resizedMask.at<cv::Vec3b>(y, x);
   if (maskPixel[0] + maskPixel[1] + maskPixel[2] > 30) {
      for (int c = 0; c < 3; c++) {
```

```
            dstRow[dstX][c] = dstRow[dstX][c] * 0.1 + maskPixel[c] * 0.9;
        }
    }
}
```

Thresholding on total brightness (sum of channels > 30) approximates transparency—very dark pixels (near-black) treated as transparent. This crude approach works adequately when proper alpha channels are unavailable.

Boundary Handling:

```
if (dstY < 0 || dstY >= dst.rows) continue;
if (dstX < 0 || dstX >= dst.cols) continue;
```

Masks may extend beyond image boundaries (face near edge). These checks prevent segmentation faults by skipping out-of-bounds pixels.

Limitations and Future Improvements:
Current implementation limitations:

1. **No rotation:** Assumes upright faces; fails for tilted heads
2. **No 3D perspective:** Mask appears flat; doesn't account for head angle
3. **Imperfect sizing:** Fixed scaling ratios don't adapt to individual face shapes
4. **No temporal smoothing:** Jittery mask movement when face detection fluctuates

Professional AR implementations address these through:

- **Facial landmark detection:** 68-point face mesh for precise alignment
- **3D head pose estimation:** Rotation matrices for perspective-correct rendering
- **Temporal filtering:** Kalman filters smooth position/scale over time
- **Machine learning:** Neural networks predict head boundaries from partial visibility

Despite limitations, the current implementation successfully overlays masks with reasonable alignment for frontal faces, demonstrating the core AR overlay pipeline.

Task 17: Custom Depth Estimator

As detailed in Task 10, I chose to implement a custom gradient-based depth estimation algorithm rather than integrating the Depth Anything V2 neural network. This decision prioritized computational efficiency and educational value over absolute depth accuracy.

Rationale:

1. **Performance:** The custom approach runs in ~2ms per frame vs. 50-100ms for DA2, leaving computational budget for additional filters while maintaining 30fps.
2. **Simplicity:** Avoiding ONNX Runtime dependency simplifies the build process, cross-platform compatibility, and deployment. The assignment's primary focus is image filtering, not deep learning integration.
3. **Sufficient Quality:** For artistic filtering effects (depth-based blur, focus effects), approximate depth cues provide adequate results. Pixel-perfect depth maps are unnecessary when the goal is aesthetic enhancement rather than precise 3D reconstruction.
4. **Educational Value:** Implementing classical computer vision heuristics demonstrates traditional approaches and their trade-offs, complementing the learning objectives around spatial filtering and real-time processing.

Trade-off Analysis:

Aspect	Custom Heuristic	Deep Learning (DA2)
Accuracy	Moderate (fails on complex scenes)	High (learned from millions of examples)
Speed	~2ms per frame	~50-100ms per frame
Dependencies	OpenCV only	ONNX Runtime, model files
Build Complexity	Simple makefile	Complex linking, runtime libraries
Failure Modes	Predictable (lighting, composition dependent)	Unpredictable (fails on out-of-distribution inputs)
Interpretability	Clear algorithmic steps	Black box neural network

For this project's scope and the graduate-level expectation of understanding algorithmic design choices, the custom approach represents a pragmatic engineering decision balancing multiple constraints.

REFLECTION

This project provided hands on experience with fundamental computer vision operations and real-time systems optimization. Key learnings include:

1. Filter Separability: Understanding that 2D convolutions can be decomposed into sequential 1D operations drastically improved my intuition for computational complexity. The 10.7× speedup from blur5x5_2 demonstrated that algorithmic choices matter more than micro-optimizations.

2. Memory Access Patterns: The performance difference between cv::Mat::at<>() and cv::Mat::ptr<>() highlighted how memory access patterns affect cache performance. Direct pointer arithmetic eliminates bounds checking and enables better compiler optimizations.

3. Color Space Understanding: Implementing custom grayscale conversions and color detection algorithms deepened my understanding of RGB color space properties and perceptual color differences. Distinguishing red objects from skin tones required careful threshold tuning based on color theory.

4. Real-time Constraints: Achieving 30fps video processing required constant attention to computational efficiency. I learned to profile code, identify bottlenecks, and make data-driven optimization decisions.

5. Face Detection Trade-offs: Working with Haar cascades illustrated classical computer vision approaches versus modern deep learning. While less accurate than CNNs, Haar cascades run faster and require no GPU, suitable for real-time applications.

6. System Integration: Managing 12+ filters with seamless keyboard switching taught me about state management, mode handling, and user interface design in interactive systems.

The most challenging aspect was the color pop effect's color detection, requiring extensive parameter tuning to balance true positives (actual colored objects) against false positives (skin tones, whites). This empirical tuning process emphasized that computer vision often requires iterative refinement rather than perfect solutions. Overall, this project solidified my understanding of spatial filtering, edge detection, and the performance considerations essential for real-time computer vision systems.

ACKNOWLEDGEMENTS

I completed this project independently as a graduate student in CS 5330. I would like to acknowledge the following resources and assistance:

Technical Resources:
- OpenCV 4.x library (docs.opencv.org) for computer vision operations, API reference, and implementation guidance
- Pre-trained Haar cascade classifier (haarcascade_frontalface_alt2.xml) from the OpenCV GitHub repository for face detection
- Standard C++ and STL for efficient memory management and real-time performance optimization
- Digital image processing and computer vision literature for theoretical foundations on convolution operations, gradient computation, and filter design
- Claude AI (Anthropic) for assistance with understanding separable filter theory and algorithmic optimization concepts

Algorithms and Methods:
- Sobel operator for edge detection (Sobel, I., 1968)
- Viola-Jones face detection algorithm using Haar cascade classifiers (Viola, P., & Jones, M., 2001)
- Gaussian blur and separable filter theory from standard computer vision literature
- Sepia tone transformation matrix coefficients from digital photography processing standards

References:

- Sobel, I. (1968). "An Isotropic 3×3 Image Gradient Operator." Presentation at Stanford Artificial Intelligence Project (SAIL).

- Viola, P., & Jones, M. (2001). "Rapid Object Detection using a Boosted Cascade of Simple Features." Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), Vol. 1, pp. 511-518.

- Sobel edge detection as described in: Shapiro, L. G., & Stockman, G. C. (2001). "Computer Vision." Prentice Hall, pp. 326-330.